

HyperLogLogLog: Cardinality Estimation With One Log More

Matti Karppa

mattk@itu.dk

BARC

IT University of Copenhagen

Rasmus Pagh

pagh@di.ku.dk

BARC

University of Copenhagen

ABSTRACT

We present HyperLogLogLog, a practical compression of the HyperLogLog sketch that compresses the sketch from $O(m \log \log n)$ bits down to $m \log_2 \log_2 \log_2 m + O(m + \log \log n)$ bits for estimating the number of distinct elements n using m registers. The algorithm works as a drop-in replacement that preserves all estimation properties of the HyperLogLog sketch, it is possible to convert back and forth between the compressed and uncompressed representations, and the compressed sketch maintains mergeability in the compressed domain. The compressed sketch can be updated in amortized constant time, assuming n is sufficiently larger than m . We provide a C++ implementation of the sketch, and show by experimental evaluation against well-known implementations by Google and Apache that our implementation provides small sketches while maintaining competitive update and merge times. Concretely, we observed approximately a 40% reduction in the sketch size. Furthermore, we obtain as a corollary a theoretical algorithm that compresses the sketch down to $m \log_2 \log_2 \log_2 \log_2 m + O(m \log \log \log m / \log \log m + \log \log n)$ bits.

CCS CONCEPTS

• Information systems → Data management systems; • Theory of computation → Design and analysis of algorithms.

KEYWORDS

distinct elements, cardinality estimation, hashing, hyperloglog

1 INTRODUCTION

Counting the number of distinct elements, or the *cardinality*, of a data stream is a basic operation that has applications in network traffic analysis [16], organization of large datasets [11], genome analysis [5], analysis of social networks [1], including large-scale industrial applications such as estimating the number of distinct Google search queries [12]. The naïve approach of storing all unique elements in the data stream quickly becomes prohibitive as the number of distinct elements in the data stream grows into the order of billions, which calls for sketching approaches that maintain a sketch of the cardinality using a limited number of bits.

Apart from the sheer number of bits, other design considerations for the sketches include idempotence of updates, that is, that repeated updates with the same element never change the sketch, control on the estimation error, efficient updates, such that the sketch can be updated even if the amount of computational resources is limited, and mergeability of sketches, such that the data stream can be processed by multiple computational units at once and the sketches can be merged to produce output identical to as if a single machine had read all the input. The HyperLogLog

sketch by Flajolet, Fusy, Gandonet, and Meunier [8] is a practical example of such a sketch and has become a standard technique for cardinality estimation. The HyperLogLog sketch makes use of m counters, or *registers*, of $O(\log \log n)$ bits to produce an estimate of the cardinality with a relative standard error of $1.04/\sqrt{m}$.

Our contribution. We show that if we store offsets from a base value in the registers and allow $O(m/\log m)$ registers to overflow, storing the overflowing registers sparsely using $O(\log m)$ bits each, then, with high probability, $m \log_2 \log_2 \log_2 m + O(m + \log \log n)$ bits suffice to represent the HyperLogLog sketch. The sketch is a true compression of the HyperLogLog sketch, preserves all estimation properties of HyperLogLog, and is amenable to the tricks that can be used to boost the estimation accuracy of HyperLogLog sketches. Furthermore, the sketch preserves mergeability in the compressed domain without having to be decompressed for merging, and we show that the update time is amortized constant for n sufficiently larger than m . As a corollary, we obtain a sketch of $m \log_2 \log_2 \log_2 \log_2 m + O(m \log \log \log m / \log \log m + \log \log n)$ bits if a succinct dictionary is used for the sparse representation.

We also provide a C++ implementation of the sketch and show empirically that it is applicable as a drop-in replacement for HyperLogLog in practice and can be implemented with a competitive performance over a wide parameter range.

Related work. The HyperLogLog branch of research was initiated by Flajolet and Martin in [9] where the idea of their algorithm was to construct an $m \times w$ bit matrix using a random hash function to divide the data stream into m independent streams, corresponding to the rows of the matrix, where the high bits of the hash value select the substream, and the location of the least significant one-bit in the low-bits of the hash value determines the column; the value thus determined is set to 1. Each row then produces an estimate of the cardinality of the set, and an arithmetic mean is used to lower the variance of the estimate. In [7], this was optimized by only storing the maximum of the *locations* of the first one-bits, requiring $\log \log n$ bits, resulting in the LogLog algorithm and its more practical variant, the SuperLogLog. Eventually, this led to HyperLogLog [8], the key change of which was the use of the harmonic mean instead of the arithmetic mean for lower variance.

The HyperLogLog sketch has seen a lot of practical applications and improvements, including but not limited to the HyperLogLog++ [12] that includes practical engineering work with numerous tricks for reducing memory usage and improving the accuracy of the estimates, and the HLL-Tailcut [21] where the registers store the offsets with respect to the minimum register value, and overflowing values are discarded.

Other approaches to counting distinct elements include Linear Counting [20], and MinCount [2, 10] that was used by Google before being replaced with HyperLogLog++ [12]. There exist a number of

non-mergeable methods for cardinality estimation in recent literature that offer potentially better space–accuracy tradeoffs than their mergeable counterparts, and also methods to convert mergeable sketches into more efficient sketches at the cost of non-mergeability. See Pettie, Wang, and Yin [19] for an overview and analysis of such methods, along with their *Curtain* sketch.

The HyperLogLog sketch is known to be non-optimal in space usage. Both the Flajolet-Martin [9] and the HyperLogLog sketch [8] are known to have an entropy of $O(m)$ bits [19], suggesting the application of entropy compression methods to achieve the lower bound, although at the expense of losing constant time updates. In [15], Lang applies entropy compression to the original Flajolet-Martin sketch [9] to get a practical sketch below the entropy of the HyperLogLog sketch. In [13], Kane, Nelson, and Woodruff present a theoretical algorithm that achieves the optimal $O(m)$ space bound with $O(1)$ updates. This was further improved by Błasiok [4] by reducing the amount of space required for parallel repetitions of the algorithm for reducing the probability of failure. However, we are not aware of practical implementations of these optimal algorithms, and regard them as mainly of theoretical interest.¹

Organization. The remainder of this paper is organized as follows. In Section 2, we present the mathematical notation and preliminaries used in the remainder of the paper. In Section 3, we recap HyperLogLog and present our modifications that yield the HyperLogLogLog algorithm, prove its space usage, and present changes that make the algorithm practical. In Section 4 we present an overview of our implementation and engineering choices. Finally, in Section 5 we report on an empirical study where we show that our algorithm is practical in terms of runtime and provides space-efficient sketches that yield estimates equal to those of HyperLogLog, and compare the running times and sketch sizes to other well-known implementations.

2 PRELIMINARIES

We write $[n] = \{1, 2, \dots, n\}$. We say that w is the *word size*, meaning that it matches the number of bits output by our hash functions and is somehow an efficient unit of data as processed by an underlying computer architecture. When the actual implementations are concerned, we are implicitly assuming $w = 64$ unless noted otherwise. All logarithms are base 2 unless otherwise stated, and $\ln$ is the natural logarithm.

We denote the universe over which we want to count the number of distinct elements by $\mathcal{U}$. In our experimental work, $\mathcal{U}$ is either a subset of integers or ASCII strings. We assume we have access to a family of random hash functions $h : \mathcal{U} \rightarrow [2^w]$ that map the elements of the universe to w -bit integers uniformly at random.

When we say that $M \in U^m$ is an *array*, we mean that M consists of m elements from the set U , and denote the lookup of the j th element by $M[j]$ and the assignment of element u to the j th position by $M[j] \leftarrow u$.

When we say that $S \subseteq K \times V$ is an *associative array*, we mean that S is a variable-sized data structure that stores tuples $(k, v) \in K \times V$,

¹A particular difficulty in implementing these constructions is the use of *Variable-Bit-Length Arrays* to store an array of variable-length counters compactly. The construction of Blandford and Blelloch [3] uses constant time per operation, but is complicated and a direct implementation appears to use space that is at least 3 times larger than the entropy of the counters.

or equivalently keys $k \in K$ that map to values $v \in V$. We assume S supports the operations of membership query $k \in S$ to test whether the key k has an associated value in S , retrieval of values by key $S[k]$, assigning the value v to the key k by $S[k] \leftarrow v$, and removing a key-value pair $\text{del } S[k]$. We tacitly assume that there is only ever at most one value associated with any particular key, and that the assignment of a pre-existing key with a new value replaces the old value, so we treat S as a map.

The function $\rho : [2^w] \rightarrow [w]$ is defined to be the 1-based index of the first one-bit in the bit representation of the (w -bit) integer. That is, for the bit sequence with $k - 1$ initial zeros $\rho(0^{k-1}1) = k$. We leave $\rho(0)$ undefined.

We say that random variables $X_1, X_2, \dots, X_m$ are *negatively dependent* if, for all $i \neq j$, they satisfy $E[X_i X_j] \leq E[X_i]E[X_j]$. We say that a random event happens *with high probability* if we can choose a constant $0 < \beta < 1$ such that the event happens with probability $1 - \beta$. We use the Iverson bracket notation to denote an indicator variable $\llbracket \varphi \rrbracket$ that receives 1 if and only if the expression φ is true and 0 otherwise. We need the following form of the Chernoff bound.

LEMMA 2.1 (CHERNOFF [6, EQUATION 1.7]). *Let $X = \sum_{j \in [m]} X_j$ be a sum of negatively dependent variables with all $X_j \in [0, 1]$. Then, for all $\epsilon > 0$,*

$$\Pr[X > (1 + \epsilon)E[X]] \leq \exp\left(-\frac{\epsilon^2}{3}E[X]\right).$$

3 THE HYPERLOGLOG LOG ALGORITHM

3.1 Problem statement

We formally state the problem as follows. Given a sequence of elements $(y_1, y_2, \dots, y_s)$ from a universe $\mathcal{U}$, determine the number of distinct elements $n = |\{y_i : i \in [s]\}|$. Unless otherwise stated, we will assume that n is the correct (but unknown) cardinality of the datastream in question.

3.2 HyperLogLog recap

Since our algorithm is a modification on HyperLogLog, it is necessary to understand how HyperLogLog works, so we start by restating the HyperLogLog algorithm [8] using our notation. The underlying idea of the algorithm is that, if we hash the elements uniformly at random, we expect to see exactly one half of the elements begin with a 1, exactly one fourth begin with the bitsequence 01, one eighth begin with the bitsequence 001, and so on; so the largest position r of the first one-bit witnessed over the sequence is a (weak) indication that the stream has a cardinality of approximately 2^r .

Let us assume that we work over a universe $\mathcal{U}$. The HyperLogLog data structure consists of an array $M \in [w]^m$ of m elements of $\log w$ bits where m is a power of two.² Furthermore, we fix two³ random hash functions $h : \mathcal{U} \rightarrow [2^w]$ and $f : \mathcal{U} \rightarrow [m]$ by drawing them from the family of random hash functions. Initially we set all elements of M to be zeros. We say that the elements of M are *registers*. We make the observation that $w = \Omega(\log n)$ by the pigeon

²This requirement is not strictly necessary, but in practice the way the algorithm implemented, this is a reasonable assumption.

³In the original exposition [8], only one 32-bit hash function was used and the most significant bits were used for register selection and the least significant bits as input for the function ρ ; this can be seen as a special case of our treatment.

hole principle, as otherwise hash collisions necessarily mask some of the distinct elements.

The data structure supports three operations: `UPDATE(y)` that updates the data structure with respect to the element $y \in \mathcal{U}$, `ESTIMATE()` that returns the present cardinality estimate, and `MERGE(M1, M2)` that takes two sketches as input and returns a third sketch that corresponds to the sketch that would have been obtained if the elements that were used to update the individual sketches had been directly applied on the output sketch—assuming the two input sketches have been constructed using the same hash functions.

Updates are performed by selecting the index of a register by computing $j = f(y)$, and then update the value of the register to be the maximum of its present value and the value $\rho(h(y))$. That is, the invariant maintained by the algorithm is that the register $M[j]$ holds the maximum ρ -value over the elements assigned to the j th substream by the hash function f . Since the hash values are w bits long, we have $0 \leq \rho(h(y)) < w$, so the values can be represented with $\log w = \Theta(\log \log n)$ bits.

To construct an estimate from the register values, we compute the bias-corrected, normalized harmonic mean of the estimates for the substreams

$$E = \alpha_m m^2 \cdot \left(\sum_{j=1}^m 2^{-M[j]} \right)^{-1},$$

where the bias-correction term α_m satisfies $\alpha_{16} = 0.673$, $\alpha_{32} = 0.697$, $\alpha_{64} = 0.709$, and $\alpha_m = 0.7213/(1 + 1.079/m)$ for $m \geq 128$; see [8] for the details. Finally, the algorithm includes further bias-correction by applying Linear Counting for small cardinality ranges, and a similar correction for large ranges.

Merging is very simple: simply construct a new array the elements of which are the elementwise maxima of the input sketches. These procedures are presented in pseudocode in Algorithm 1. The resulting estimate has low bias (but is not unbiased), and it can be shown that the relative standard error of the estimate is approximately $1.04/\sqrt{m}$ [8].

3.3 Asymptotic argument

It is well-known⁴ that $O(m)$ bits suffice for an entropy-compressed version of HyperLogLog. However, entropy-compression is expensive and does not allow for efficient updates. We will show that by using $O(\log \log n)$ bits to store a *base* value B and an array of offsets M as a dense array, plus a limited number of $(j, M[j])$ pairs as a *sparse* associative array, $m \log \log \log m + O(m)$ bits suffice for the sketch with high probability.

Suppose we feed n distinct values to the HyperLogLog sketch. Then each of the registers $M[j]$ can be treated as a random variable, and the distribution of the register values is determined by the following lemma.

LEMMA 3.1 ([21, APPENDIX A]). *After updating the HyperLogLog sketch of m registers with n distinct values, the distribution of each*

```

1 Let  $m$  be a power of two. Initialize array  $M \in [w]^m$  to be all
  zeros. Let  $h : \mathcal{U} \rightarrow [2^w]$  and  $f : \mathcal{U} \rightarrow [m]$  be fixed
  random hash function.
2 Procedure UPDATE( $M, y$ )
3    $x \leftarrow h(y)$ 
4    $j \leftarrow f(y)$ 
5    $M[j] \leftarrow \max\{M[j], \rho(x)\}$ 
6 End Procedure
7 Procedure ESTIMATE( $M$ )
8    $E \leftarrow \alpha_m m^2 \cdot \left( \sum_{j=1}^m 2^{-M[j]} \right)^{-1}$ 
9    $V \leftarrow |\{j : M[j] = 0\}|$ 
10  return  $\begin{cases} m \ln \frac{m}{V} & \text{if } E \leq \frac{5}{2}m \text{ and } V \neq 0 \\ -2^{32} \ln \left( 1 - \frac{E}{2^{32}} \right) & \text{if } E > \frac{2^{32}}{30} \\ E & \text{otherwise} \end{cases}$ 
11 End Procedure
12 Procedure MERGE( $M_1, M_2$ )
13   Initialize  $M \in [w]^m$ 
14   for  $j \in [m]$  do
15      $M[j] \leftarrow \max\{M_1[j], M_2[j]\}$ 
16   end
17   return  $M$ 
18 End Procedure

```

Algorithm 1: HyperLogLog.

$M[j]$ satisfies

$$\Pr[M[j] = k] = \begin{cases} \left(1 - \frac{1}{m}\right)^n & \text{if } k = 0, \\ \left(1 - \frac{1}{m2^k}\right)^n - \left(1 - \frac{1}{m2^{k-1}}\right)^n & \text{otherwise.} \end{cases}$$

COROLLARY 3.2. *The distribution of $M[j]$ satisfies*

$$\Pr[M[j] \leq k] = \left(1 - \frac{1}{m2^k}\right)^n.$$

Note that Lemma 3.1 only applies to any particular register j , but not to all as a whole, since the registers are negatively dependent. Throughout this section, we are going to treat the register values as real-valued, continuous random variables, as the map $k \mapsto 1 - \frac{1}{m2^k}$ is monotone, only rounding to integers at the very end. We also need the following lemma.

LEMMA 3.3. *Let $B = \log \frac{n}{m}$. Then, for $\Delta \in (0, B)$ and each $j \in [m]$,*

- $\Pr[M[j] < B - \Delta] < \exp(-2^\Delta) < 2^{-\Delta}$, and
- $\Pr[M[j] > B + \Delta] < 2^{-\Delta}$.

PROOF. It is well-known that $(1 - x)^n < \exp(-nx)$ for all $0 < x < 1$. By Corollary 3.2 and treating k as a real variable,

$$\begin{aligned} \Pr[M[j] < B - \Delta] &= \left(1 - \frac{1}{m2^{B-\Delta}}\right)^n \\ &< \exp\left(-\frac{n}{m2^{B-\Delta}}\right) = \exp(-2^\Delta) < 2^{-\Delta}. \end{aligned}$$

⁴As mentioned, for example, in [19]. Lang [15] suggests that the constant is just below 2.832 which is supported by numerical experiments with Lemma 3.1.

Likewise, by complement from Corollary 3.2 and by approximating $(1-x)^n < 1-nx$ for $0 < x < 1$,

$$\begin{aligned} \Pr[M[j] > B + \Delta] &= 1 - \left(1 - \frac{1}{m^{2^{B+\Delta}}}\right)^n \\ &< \frac{n}{m^{2^{B+\Delta}}} = 2^{-\Delta}. \end{aligned}$$

□

We will start by showing that, for sufficiently large n and m , the sketch can be represented with $O(m \log \log m + \log \log n)$ bits, with high probability.

THEOREM 3.4. *For $\beta \in (0, 1)$ and $n > 2m^2/\beta$, all register values can be represented as offsets from the base value $B = \log \frac{n}{m}$ using at most $\lceil \log \log \frac{2m}{\beta} \rceil + 1 = O(\log \log m)$ bits, with probability at least $1 - \beta$.*

PROOF. Fix the constant $0 < \beta < 1$. We will set $\Delta = \log \frac{2m}{\beta}$. By our condition on n , $\Delta < B$, so we apply Lemma 3.3, and get, for each $j \in [m]$, $\Pr[M[j] \notin (B - \Delta, B + \Delta)] < 2 \cdot 2^{-\Delta} = \frac{\beta}{m}$. By the union bound over the m registers, we get that all registers are within this interval with probability at least $1 - \beta$. Finally, by our choice of Δ , we can encode an integer in the desired range of $(B - \Delta, B + \Delta)$ by using at most $\log(2\Delta) = \lceil \log \log \frac{2m}{\beta} \rceil + 1$ bits. □

Theorem 3.4 is similar to the HLL-Tailcut approach of [21], and could indeed be used to show that the tailcut approach requires asymptotically only few bits. However, we can do better than this and use only $\log \log \log m + O(1)$ bits per register by showing that $O(m)$ bits suffice for representing the overflowing registers sparsely as $(j, M[j])$ pairs.

THEOREM 3.5. *Suppose $n > 4m \log m$. Then at least $m - \frac{m}{\log m}$ register values can be represented as offsets from the base value $B = \log \frac{n}{m}$ using at most $\lceil \log(2 + \log \log m) \rceil + 1 = \log \log \log m + O(1)$ bits, with probability at least $1 - \exp(-m/(6 \log m))$.*

PROOF. We will set $\Delta = 2 + \log \log m$. By our condition on n , $\Delta < B$, so we can apply Lemma 3.3. For each $j \in [m]$, we define an indicator variable $X_j = \mathbb{I}[M[j] \notin (B - \Delta, B + \Delta)]$. We note that $X_j \sim \text{Bernoulli}(p)$. We get from Lemma 3.3 that p satisfies

$$p = \Pr[X_j = 1] < 2 \cdot 2^{-\Delta} = \frac{1}{2 \log m}.$$

We then define the sum variable $X = \sum_{j \in [m]} X_j$ and observe that, by linearity of expectation, $E[X] < m/(2 \log m)$. Since the variables X_j are negatively dependent, we can apply the Chernoff bound of Lemma 2.1 to bound the number of registers whose values do not fall in our desired interval

$$\Pr\left[X > \frac{m}{\log m}\right] \leq \exp\left(-\frac{m}{6 \log m}\right).$$

Finally, we note that by our choice of Δ , any integer in $(B - \Delta, B + \Delta)$ can be encoded using $\lceil \log(2\Delta) \rceil = \lceil \log(2 + \log \log m) \rceil + 1 = \log \log \log m + O(1)$ bits, and the sparse representation requires $O\left(\frac{m}{\log m}\right) \cdot O(\log m) = O(m)$ bits by the fact that the register indices are in $[m]$ and Theorem 3.4. □

As a corollary of Theorem 3.5, we obtain an algorithm that we might call HyperLogLogLogLog that uses $m \log \log \log \log m + O(m \log \log \log m / \log \log m + \log \log n)$ bits for the sketch if we use a succinct dictionary for the sparse representation. We believe this algorithm is only a theoretical curiosity, but present it for completeness.

COROLLARY 3.6. *Suppose $n > 4m \log \log m$. Then at least $m - \frac{m}{\log \log m}$ register values can be represented as offsets from the base value $B = \log \frac{n}{m}$ using at most*

$$\lceil \log(2 + \log \log \log m) \rceil + 1 = \log \log \log \log m + O(1)$$

bits, with probability at least $1 - \exp(-m/(6 \log \log m))$. Using a succinct dictionary for the sparse representation, the sketch has a total size of $m \log \log \log \log m + O(m \log \log \log m / \log \log m + \log \log n)$ bits.

PROOF. The proof is otherwise the same as in Theorem 3.5 except we use $\Delta = 2 + \log \log \log m$. For the sparse representation, we note that it is known that succinct dictionaries (see, for example, [17]) require $O(s \log \frac{m}{s})$ bits where s is the number of distinct elements in the dictionary and m is the size of the universe. In particular, the same m as in our case works since the elements are pairs in $[m] \times [\log m]$ by Theorem 3.4, with high probability, so the number of bits required in the standard representation for any element in the universe is $O(\log m)$. Applying the bound on the succinct dictionary size, together with $s = m / \log \log m$, by our choice of Δ , and the number of bits required for the base value, we get the bound of $m \log \log \log \log m + O(m \log \log \log m / \log \log m + \log \log n)$ bits on the total size of the sketch. □

Finally, we note that storing the base value B requires $O(\log \log n)$ bits which we can afford since we are going to need $O(\log n)$ auxiliary space for storing the computed hash values anyway. This is also reflected in the more precise bound of [13].

3.4 Practical algorithm

The practical applicability of Theorems 3.5 is somewhat limited as the $\log \log \log m$ is an extremely slowly growing function. Indeed, it is commonly believed that there are roughly 10^{80} atoms in the observable universe, and $\log \log \log 10^{80} < 3.01$ which is hardly a large number. Furthermore, the proofs only address the size of the resulting sketch and make use of n which we cannot do in practice.

However, numerical simulations reveal that using 3 bits per register tends to yield small sketches for practical values of m and n , and so the intuition of the sparse/dense split seems well applicable in practice. We will introduce a separate parameter κ that controls the number of bits per dense register. For our implementation, we fix $\kappa = 3$ (along with $w = 64$). This is quite close to the entropy bound of 2.832 [15], so we cannot even hope to do much better.

Let $M' \in [w]^m$ be the registers of the uncompressed HyperLogLog sketch. We denote the fixed-size dense array of offsets by $M \in [\kappa]^m$, and the base value $B \in [w]$. The dense part corresponds to the “fat region” of the distribution of values around the expectation, and the registers in the fat region satisfy

$$B \leq M'[j] < B + 2^\kappa, \quad (1)$$

and we store $M[j] = M'[j] - B$ using κ bits. The sparse part corresponds to registers j that do *not* satisfy Equation (1), and they are stored in an associative array $S \subseteq [m] \times [w]$ whose elements are (j, r) pairs where $r = M'[j]$, or equivalently, S maps the sparsely represented j to their corresponding register values. Each element of S takes $\log m + \log w$ bits.

Thus, the HyperLogLogLog data structure is a 3-tuple (S, M, B) , and initially $S = \emptyset$, M is initialized to all-zeros, and $B = 0$. The size of the data structure is variable, and is determined unambiguously by the number of elements stored in S , or equivalently, the choice of the base value B . The total size of the data structure is

$$m\kappa + |S| \cdot (\log m + \log w).$$

If B is chosen so as to minimize the number of bits required, we say that *the sparse-dense split is optimal*.

After each update, if any of the register values was changed, we maintain as an invariant that the split is optimal by running a subroutine which we call `COMPRESS()`. The subroutine determines the optimal B' , and if this differs from the present B , performs a `REBASE()` operation whereby registers are reassigned into sparse or dense representations, depending on which side of the split determined by B' they fall.

The `UPDATE()` procedure is shown as pseudocode in Algorithm 2. The full runtimes of each operation depends on the actual choice of data structures, but it is immediate that the compression routine is rather expensive: we need to try up to w different new base values, and evaluating the size of the resulting data structure requires $\Omega(m)$ operations for each different proposed new base value, so the compression routine requires at least $\Omega(mw)$ operations. However, it is easy to see that if the n is sufficiently larger than m , then only very few elements actually trigger an update; thus, as the next lemma shows, the update times are actually amortized constant over a sufficiently large n . We note that Lemma 3.7 is not tight, as it makes some rather crude and pessimistic approximations.

LEMMA 3.7. *For sufficiently large n satisfying $n/(\log n)^2 > m^2$, the updates are amortized constant, with high probability.*

PROOF. Fix a constant $\gamma \geq 1$. From Corollary 3.2 and by approximating $(1 - x)^n \geq 1 - nx$, we get for each j that

$$\Pr[M[j] > k] \leq \frac{n}{m2^k}.$$

Let k be the largest value that we expect to see. Setting $k = \log(\gamma n)$, we get by the union bound that all registers are below this value at probability $1 - 1/\gamma$.

Let us then consider the total amount of work for the compressions. The compression is triggered at most km times. Each time the compression routine is triggered, the amount of work required is $O(km)$ since we need to try at most k base values and perform $O(m)$ work for each base value candidate. The total amount of compression work is thus $O(m^2 k^2) = O(m^2 (\log(\gamma n))^2)$. In addition to compressions, we also need $O(1)$ work for every update. Average work for n distinct elements is thus $\frac{O(n) + O(m^2 (\log n)^2)}{n} = O(1)$ for sufficiently large n by our assumption that $n/(\log n)^2 > m^2$, thus an amortized constant. $\square$

The other supported operations are the regular HyperLogLog operations `ESTIMATE()`, and `MERGE()`. Estimation is performed exactly

1 Let m be a power of two. Initialize array $M \in [\kappa]^m$ to be all zeros, the associative array $S \subseteq [m] \times [w]$ to be $S \leftarrow \emptyset$, and the base $B \leftarrow 0$. Let $h : \mathcal{U} \rightarrow [2^w]$ and $f : \mathcal{U} \rightarrow [m]$ be fixed random hash function.

2 **Procedure** `GET`(S, M, B, j)
3 **return** $\begin{cases} S[j] & \text{if } j \in S \\ B + M[j] & \text{otherwise} \end{cases}$

4 **End Procedure**

5 **Procedure** `UPDATE`(S, M, B, y)

6 $j \leftarrow f(y)$
7 $r \leftarrow \rho(h(y))$
8 $r_0 \leftarrow \text{GET}(S, M, B, j)$
9 **if** $r > r_0$ **then**
10 **if** $B \leq r < B + 2^\kappa$ **then**
11 $M[j] \leftarrow r - B$
12 **if** $j \in S$ **then**
13 $\text{del } S[j]$
14 **end**
15 **else**
16 $S[k] \leftarrow r$
17 **end**
18 `COMPRESS`(S, M, B)
19 **end**

20 **End Procedure**

21 **Procedure** `COMPRESS`(S, M, B)

22 $B' \leftarrow \arg\max_{B' \in [w]} |\{j : B'' \leq \text{GET}(S, M, B, j) < B'' + 2^\kappa\}|$
23 **if** $B' \neq B$ **then**
24 `REBASE`(S, M, B, B')
25 **end**

26 **End Procedure**

27 **Procedure** `REBASE`(S, M, B, B')

28 **for** $j \in [m]$ **do**
29 $r \leftarrow \text{GET}(S, M, B, j)$
30 **if** $B' \leq r < B' + 2^\kappa$ **then**
31 $M[j] \leftarrow r - B'$
32 **if** $j \in S$ **then**
33 $\text{del } S[j]$
34 **end**
35 **else**
36 $S[j] \leftarrow r$
37 **end**
38 **end**
39 $B \leftarrow B'$

40 **End Procedure**

Algorithm 2: HyperLogLogLog update procedure.

as in the case of HyperLogLog, with the exception that instead of array access, we need to use the auxiliary `GET()` function to access the elements. The `MERGE()` operation is performed in the compressed domain, so at no point is there need to perform a full decompression into the regular HyperLogLog representation. The `MERGE()` operation is presented as pseudocode in Algorithm 3.

```

1 Procedure MERGE( $S_1, M_1, B_1, S_2, M_2, B_2$ )
2   Initialize  $S \leftarrow \emptyset$ ,  $M \in [2^K]^m$  as zeros, and
    $B \leftarrow \max\{B_1, B_2\}$ 
3   for  $j \in [m]$  do
4      $r_1 \leftarrow \text{GET}(S_1, M_1, B_1)$ 
5      $r_2 \leftarrow \text{GET}(S_2, M_2, B_2)$ 
6      $r \leftarrow \max\{r_1, r_2\}$ 
7     if  $B \leq r < B + 2^K$  then
8        $M[j] \leftarrow r - B$ 
9     else
10       $S[j] \leftarrow r$ 
11    end
12  end
13  COMPRESS( $S, M, B$ )
14  return ( $S, M, B$ )
15 End Procedure

```

Algorithm 3: HyperLogLogLog merge procedure.

3.5 Performance improvements and heuristics

There are several tricks that can be done to improve the practical performance of the HyperLogLogLog algorithm. Some of these tricks have effects on theoretical guarantees, some don't, and others are simply a matter of implementation choices.

Importantly, choosing suitable data structures enables cache-friendly linear access through the entire HyperLogLogLog structure, and we need not actually use a potentially expensive GET() function in the COMPRESS(), REBASE(), or MERGE() subroutines.

Furthermore, the updates are determined by pairs of random variables (j, r) where j is distributed uniformly over $[m]$ and r follows a geometric distribution. This means that in most cases, *no update* takes place after seeing sufficiently many elements, as the vast majority of the r values encountered are very small. If we use $\log w$ bits to store the minimum register value, we can terminate the UPDATE() process early without any effect on theoretical guarantees, without having to even look up the actual register value.

Also, it is obvious that any reasonable base value B should be equal to an actual register value, so we need not try all w options; also, trying candidate base values in an ascending order enables us to maintain a lower bound on the number of sparse elements, which leads to a possible early termination of the COMPRESS() routine without loss of theoretical guarantees. We include these optimizations in our implementation of HyperLogLogLog.

Considering the behavior of the random (j, r) pairs, we see that, in general, the behavior is rather benign,⁵ and trying all possible base values in the COMPRESS() routine is mostly useless, as even though it is possible to construct nasty corner cases that require unexpected rebasing operations to maintain optimality, these seldom occur in practice with random data. Therefore, we implement the following changes to a variant of the algorithm we call HyperLogLogLog*: we only call COMPRESS() if the size of the data

structure needs to be increased (that is, a new element needs to be added into S), and we only try the next possible register value that is larger than B , and omit all other choices. Experiments show that these heuristics have little effect on the actual size of the resulting data structure, but they improve the actual runtimes.

We also provide as a baseline a variant which we call HyperLogLogLogB that fixes B to be the minimum register value, and maintains a counter that records the number of minimum-valued registers. Functionally, this variant behaves like HLL-Tailcut [21], except instead of allowing for error when the register values overflow, it stores them sparsely. While this is the fastest variant, as the compression becomes nearly trivial, experiments show that the sketches are noticeably larger.

4 IMPLEMENTATION

4.1 Overview

We provide a C++ implementation of the HyperLogLogLog, HyperLogLogLog*, and HyperLogLogLogB algorithms, along with a comparable implementation of the vanilla HyperLogLog, and an entropy-compressed version of the HyperLogLog, compressed using the Zstandard library.⁶ We also provide our full experimental pipeline that can be built as a Docker container for reproducibility. The code is available online⁷ under the MIT license. Functionally, our implementation is a drop-in replacement for HyperLogLog and enables conversion between the compressed and uncompressed representations, whilst producing the exact same estimates.

The guiding principles for constructing the implementation have been to minimize the memory usage and enable cache-friendly linear access through the data structure, at the cost of potentially losing optimization opportunities that would require auxiliary space. Our implementation assumes a 64-bit environment, which is reflected in the design choices for the data structures and hash functions.

4.2 Data structures

We implement both the array M and the associative array S as bit-packed arrays of 64-bit words. An array of n elements of size s bits will occupy $\lceil \frac{ns}{64} \rceil \cdot 64$ bits of memory; importantly, we allow elements to cross word boundaries. For the HyperLogLog implementation, the array M consists of 6-bit elements, so the array size is $\lceil \frac{6m}{64} \rceil \cdot 64$ bits, and for HyperLogLogLog, we use 3 bits per element, so the array size is $\lceil \frac{3m}{64} \rceil \cdot 64$ bits.

We implement the associative array by concatenating the key-value pairs into integers where the more significant bits are occupied by the key and the less significant bits by the value. We maintain the bit-packed array sorted in ascending order by applying insertion sort after each insertion. We perform lookups by binary search in $O(\log |S|)$ time. For the HyperLogLogLog associative array S , the elements are of size $\log m + 6$, since we use 64-bit hash functions. Maintaining the associative array S sorted enables linear iteration over the entire HyperLogLogLog data structure.

⁵This is assuming that we are *not* dealing with an adversary who has access to the hash function or can affect the coin tosses when selecting the hash function; in this setting, a worst-case input can cause the algorithm to fail catastrophically. This is, however, a well-known property inherent to HyperLogLog itself, as the algorithm is not robust against adversarial input [18].

⁶<https://facebook.github.io/zstd/>

⁷<https://github.com/mkarpapa/hyperlogloglog>

Table 1: Implementations used in experiments.

Name	Description	Language	Notes
HLL	Our HyperLogLog	C++	6 bits per element
HLLL	Our HyperLogLogLog	C++	Sorted array
HLLL*	Our HyperLogLogLog*	C++	Includes heuristics
HLLLB	Our HyperLogLogLogB	C++	B is minimum
HLLZ	Zstd-compressed HLL	C++, C	Zstd is in C
AHLL8	Apache HyperLogLog	C++	8 bits per element
AHLL6	Apache HyperLogLog	C++	6 bits per element
AHLL4	Apache HyperLogLog	C++	4 bits per element
ACPC	Apache CPC	C++	Compressed Probability Counting
Zeta	Google ZetaSketch	Java	HyperLogLog++

4.3 Hash functions

Our implementation accepts unsigned 64-bit integers and 8-bit byte strings as input. We also accept raw $(j, r) \in [m] \times [w]$ integer pairs as input for evaluating runtimes without hash function evaluations.

For the hash function $h : \mathcal{U} \rightarrow [2^w]$, we use Google’s Farmhash.⁸ Specifically we use the function Fingerprint for 64-bit integers, and Hash64 for strings.

We derive the hash function $f : \mathcal{U} \rightarrow [m]$ from h by applying Fibonacci Hashing [14, Section 6.4] on the hash value $h(y)$, that is, we multiply the hash value by $9e3779b97f4a7c15_{16}$ modulo 2^{64} and take the $\log m$ most significant bits of the result.

4.4 Entropy compression

We have implemented an entropy-compressed version of HyperLogLog as a baseline. The implementation behaves like vanilla HyperLogLog, but the array is compressed with Zstandard library after each update. The number of decompressions is limited by maintaining the information about the minimum value in a separate variable, much like in the case of our HyperLogLogLog implementation, and decompression is only triggered if the encountered ρ -value is at least as large as the minimum value in the registers. The idea of using entropy compression is an obvious alternative, and we use Zstandard with compression level set to 1, as suggested in [15].

5 EXPERIMENTS

5.1 Experimental setup

We evaluate a number of implementations by varying the number of registers m as powers of two from 2^4 to 2^{18} , and generate random input of n elements for $n = 2^{\frac{i}{2}}$ for $i = \{4, 5, \dots, 60\}$, that is, n is the power of a square root of 2 from 2^4 to 2^{30} , rounded to the nearest integer. The data consists of random unsigned 64-bit integers, 8-character-long alphanumeric random ASCII strings, and (j, r) -pairs where j has been drawn from $[m]$ uniformly at random and $r \sim \text{Geom}(0.5)$. The (j, r) input is only supported by our own implementations. We also evaluate our hash functions separately, without performing a full update sequence.

We compare our implementations against Apache DataSketches⁹ implementation of HyperLogLog, Apache DataSketches implementation of Compressed Probability Counting (CPC) [15], and Google’s ZetaSketch¹⁰ implementation of HyperLogLog++ [12]. The implementations are listed in Table 1.

All experiments are run in a Docker container that is managed by a Python script. Data is generated by a separate C++ program, and the data is then passed to a wrapper program that stores the data in RAM, so as to avoid I/O issues.

There are two types of experiments: update experiments and merge experiments. In the update case, the wrapper constructs a sketch by feeding the data one element at a time. Once the sketch has been constructed, the wrapper reports the time it took to construct the sketch, the cardinality estimate computed from the sketch, and bit size of the sketch. For the merge experiment, we first divide the data into two equal-sized chunks and construct one sketch for each chunk. We then report the time it takes to construct the merged sketch from the two sketches. In all cases, 10 independent repetitions were performed with different data, but the same data supplied for each implementation at a given m , n , and repetition.

The reported bit sizes for our implementations are $6m$ for HLL, 8 times the number of bytes occupied by the Zstd compressed register array for HLLZ, and $|S| \cdot (\log m + 6) + 3m$ for HLLL, HLLL*, and HLLLB. For Zeta, following the documentation of the library, we report $8m$. For Apache sketches, we use the bit size of the output of the serialization function to upper bound the size of the sketch.

The experiments were run on a computer running two Intel Xeon E5-2690v4 CPUs at 2.60 GHz with a total of 28 cores, and a total of 512 GiB of RAM, running Ubuntu 18.04.5 LTS. The experiments were run using CPython 3.8.10 and Docker 20.10.7. The Docker environment ran Ubuntu 20.04.3 LTS, and the C++ code was compiled with GCC 10.3.0. ZetaSketch was compiled with Gradle 7.2 and OpenJDK 11.0.13. We ran up to 14 experiments in parallel.

5.2 Results

The scaling of update times is reported in Figure 1 at fixed $m = 2^{15}$ registers. We report the average time per distinct element for constructing the sketch. This shows the general behavior of the algorithm: HLL is clearly the fastest, but HLLL* catches up as the number of distinct elements grows. For another view, Figure 2 records the total time for constructing a sketch with $n = 2^{30}$ distinct elements, as a function of m . HLLL and HLLL* remain competitive with vanilla HyperLogLog until $m = 2^{12}$ and $m = 2^{15}$, respectively, and then the total time starts to grow.

Figure 3 shows yet another view into this behavior in terms of load factor n/m . The figure shows the update time per distinct element with HLLL as a function of the load factor. Input data consists of (j, r) pairs, so no hashing is performed. Furthermore, the data-points indicate the number of registers used. We see that when the number of distinct elements is sufficiently large with respect to the number of registers, the algorithm achieves amortized constant behavior. Indeed, this is to be expected, as once the registers are filled by distinct values, very few updates take place, as per Lemma 3.7. For HLLL, in this particular case, this regime is reached when the

⁸<https://github.com/google/farmhash>

⁹<https://datasketches.apache.org/>

¹⁰<https://github.com/google/zetasketch>

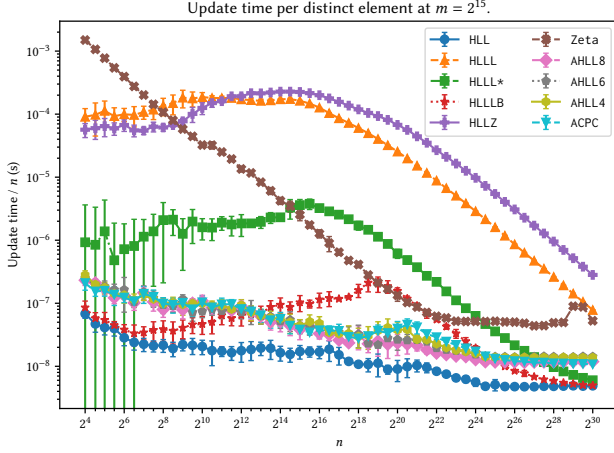


Figure 1: Scaling of update times of different algorithms at $m = 2^{15}$. The times are given in seconds / distinct element. Random unsigned 64-bit integers were used as input.

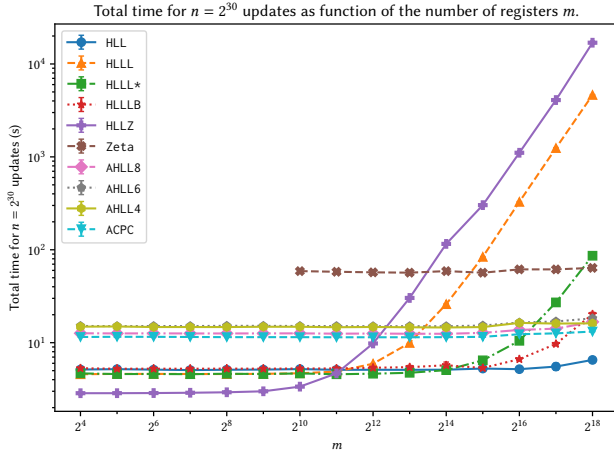


Figure 2: Total times for $n = 2^{30}$ updates as a function of m in seconds. Random unsigned 64-bit integers are used as input.

load factor is approximately 2^{17} ; this is only achieved with $m \leq 2^{13}$ due to fixed n . HLLL* reaches the same effect at a lower load factor, but the results are more noisy, owing to the heuristics, but we have omitted a separate figure for lack of space.

Although not shown here for lack of space, in particular when the stream consists of strings, hashing can make a large portion of the time. In fact, the vanilla HyperLogLog is so efficient in its updates, that almost 100% of time is spent on hashing when performing the updates with strings as input. This suggests that slower updates are not necessarily a problem, since there are other bottlenecks that may be unavoidable. In particular, I/O operations can be orders of magnitude slower, which we have deliberately avoided in our experiments by performing everything in RAM.

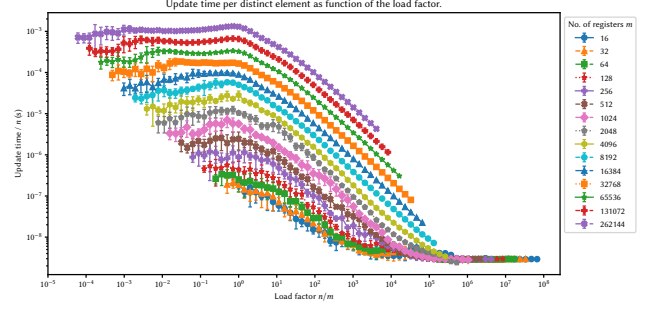


Figure 3: Update time of HLLL per distinct element as a function of the load factor n/m with different m highlighted. The times are given in seconds per element. Random (j, r) integer pairs are used as input, so no hashing is performed.

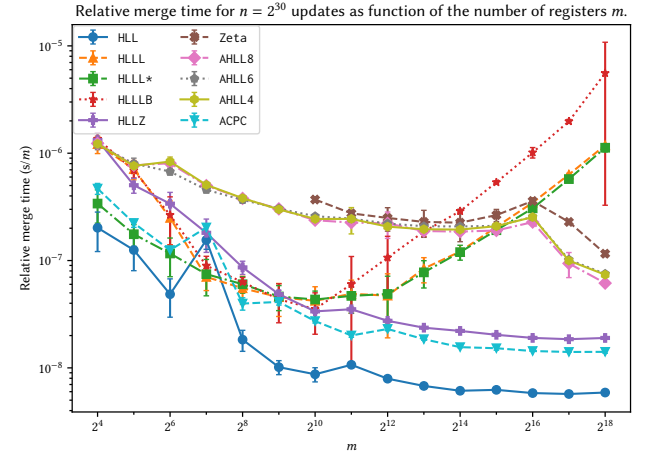


Figure 4: Relative times of merging two sketches constructed with $n = 2^{30}$ updates. The times are given in seconds/ m . Unsigned 64-bit integers were used as input.

Figure 4 shows the time required for merging of two sketches that have been constructed using $n = 2^{30}$ distinct elements, split equally in half among the sketches, as a function of the number of registers m . We see that, for HLLL, merging becomes more costly the larger the sketch size is, largely due to the application of the full compression routine after the merge. The same routine is applied also for HLLL*, so there is no discernible difference in runtime from the HLLL. Despite being slower, we are still talking about less than one third of a second for merging two $m = 2^{18}$ sketches. Perhaps surprisingly, HLLLB performs poorly. This can be explained by suboptimal choice of the base value that yields longer rebasing times, since the sparse representation is overused.

Figure 5 shows the relative sketch size as bits / m after $n = 2^{30}$ distinct element updates. In particular, this illustrates the efficiency of the heuristics we use for HLLL* since there is no discernible difference in the sketch size between HLLL and HLLL* when m is not minuscule. This figure also shows that HLLLB produces clearly

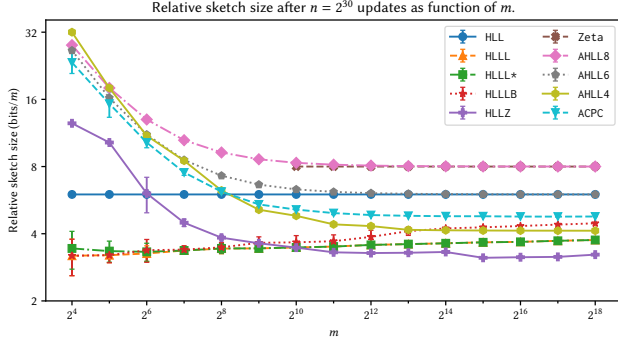


Figure 5: Relative sketch sizes (bit size / m) after $n = 2^{30}$ 64-bit integer updates as a function of m in seconds.

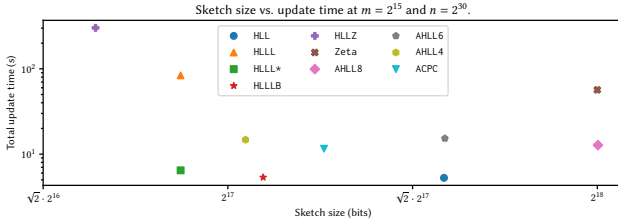


Figure 6: Update time / sketch size tradeoff plotted at $n = 2^{30}$ and $m = 2^{15}$. Unsigned 64-bit integers were used as input.

larger sketches than HLLL*. Although the sketch sizes given for the Apache DataSketches algorithms are upper bounds, the bounds appear to be quite tight as m grows. In concrete numbers, for $m = 2^{18}$, HLLL reduces the bit size of the sketch by 37.3–37.7%. On average over all choices of m , at $n = 2^{30}$, we see a reduction of over 41%.

Finally, to quantify the tradeoffs among the different implementations, Figure 6 plots the sketch size vs. update time at $n = 2^{30}$ and $m = 2^{15}$ with unsigned 64-bit integers as input. Algorithms to the left and towards the bottom would be preferred; this shows that HLLL* performs quite well in this parameter regime. In fact, this is a sweet spot for HLLL* where the sketch size is very minuscule while updates are still within the amortized constant behavior zone. Entropy compression is required for achieving smaller sketch size, at the expense of considerably higher update times.

Figure 7 provides a multi-datapoint view into the tradeoff by fixing $n = 2^{30}$ and plotting the relative sketch size vs. total update size at $m = 2^{10}, \dots, 2^{18}$. In general, if a datapoint (for fixed m , although this cannot be seen in the figure) lies to the left and to the bottom of another datapoint, it should always be preferred. This shows that there is indeed a regime where HLLL* is preferred when m is sufficiently small in relation to n , except when sketch minimality is desired, which would favor entropy compression. HLLL* is at a disadvantage when constant time updates are required at very high accuracy, but at a relatively low n .

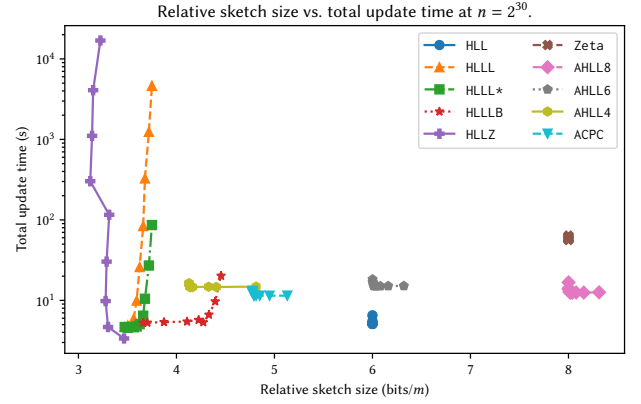


Figure 7: Update time / relative sketch size tradeoff at $n = 2^{30}$ and $m = 2^{10}, \dots, 2^{18}$ with unsigned 64-bit integers as input.

ACKNOWLEDGMENTS

We thank Martin Aumüller for constructive discussions. This work was supported by VILLUM Foundation grant 16582.

REFERENCES

- [1] Lars Backstrom, Paolo Boldi, Marco Rosa, Johan Ugander, and Sebastiano Vigna. 2012. Four degrees of separation. In *Proc. Web Science 2012*. ACM, 33–42.
- [2] Ziv Bar-Yossef, T. S. Jayram, Ravi Kumar, D. Sivakumar, and Luca Trevisan. 2002. Counting Distinct Elements in a Data Stream. In *Proc. RANDOM*. Springer, 1–10.
- [3] Daniel K. Blandford and Guy E. Blelloch. 2008. Compact dictionaries for variable-length keys and data with applications. *ACM Trans. Algorithms* 4, 2 (2008), 17:1–17:25.
- [4] Jaroslaw Blasiok. 2020. Optimal Streaming and Tracking Distinct Elements with High Probability. *ACM Trans. Algorithms* 16, 1 (2020), 3:1–3:28.
- [5] Florian P. Breitwieser, Daniel N. Baker, and Steven L. Salzberg. 2018. Kraken-Uniq: confident and fast metagenomics classification using unique k-mer counts. *Genome Biology* 19, 198 (2018).
- [6] Devdatt P. Dubhashi and Alessandro Panconesi. 2009. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, New York, NY, USA.
- [7] Marianne Durand and Philippe Flajolet. 2003. Loglog Counting of Large Cardinalities. In *Proc. ESA 2003*. Springer, 605–617.
- [8] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. 2007. HyperLogLog: the analysis of a near-optimal cardinality estimation algorithm. In *AofA: Analysis of Algorithms (DMTCS Proceedings)*. 137–156.
- [9] Philippe Flajolet and G. Nigel Martin. 1985. Probabilistic Counting Algorithms for Data Base Applications. *J. Comput. Syst. Sci.* 31, 2 (1985), 182–209.
- [10] Frédéric Giroire. 2009. Order statistics and estimating cardinalities of massive data sets. *Discret. Appl. Math.* 157, 2 (2009), 406–427.
- [11] Alon Y. Halevy, Flip Korn, Natalya Fridman Noy, Christopher Olston, Neoklis Polyzotis, Sudip Roy, and Steven Euijong Whang. 2016. Goods: Organizing Google’s Datasets. In *Proc. SIGMOD 2016*. ACM, 795–806.
- [12] Stefan Heule, Marc Nunkesser, and Alexander Hall. 2013. HyperLogLog in practice: algorithmic engineering of a state of the art cardinality estimation algorithm. In *Proc. EDBT ’13*. ACM, 683–692.
- [13] Daniel M. Kane, Jelani Nelson, and David P. Woodruff. 2010. An optimal algorithm for the distinct elements problem. In *Proc. PODS 2010*. ACM, 41–52.
- [14] Donald E. Knuth. 1998. *The Art of Computer Programming. Volume 3. Sorting and Searching*. Addison-Wesley, Boston, MA, USA.
- [15] Kevin J. Lang. 2017. Back to the Future: an Even More Nearly Optimal Cardinality Estimation Algorithm. *CoRR* abs/1708.06839 (2017). arXiv:1708.06839
- [16] Yang Liu, Wenji Chen, and Yong Guan. 2016. Identifying High-Cardinality Hosts from Network-Wide Traffic Measurements. *IEEE Trans. Dependable Secur. Comput.* 13, 5 (2016), 547–558.
- [17] Rasmus Pagh. 2001. Low Redundancy in Static Dictionaries with Constant Query Time. *SIAM J. Comput.* 31, 2 (2001), 353–363.
- [18] Kenneth G. Paterson and Mathilde Raynal. 2021. HyperLogLog: Exponentially Bad in Adversarial Settings. *IACR Cryptol. ePrint Arch.* (2021), 1139.

- [19] Seth Pettie, Dingyu Wang, and Longhui Yin. 2021. Non-Mergeable Sketching for Cardinality Estimation. In *Proc. ICALP 2021 (LIPIcs)*, Vol. 198. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 104:1–104:20.
- [20] Kyu-Young Whang, Brad T. Vander Zanden, and Howard M. Taylor. 1990. A Linear-Time Probabilistic Counting Algorithm for Database Applications. *ACM Trans. Database Syst.* 15, 2 (1990), 208–229.
- [21] Qingjun Xiao, Shigang Chen, You Zhou, and Junzhou Luo. 2020. Estimating Cardinality for Arbitrarily Large Data Stream With Improved Memory Efficiency. *IEEE/ACM Trans. Netw.* 28, 2 (2020), 433–446.